

claude-windows / 9f6357d8-0e35-491a-a830-052e094c7f37

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\9f6357d8-0e35-491a-a830-052e094c7f37\subagents\workflows\wf\_635348d6-51c\agent-ac8359a2d1de47b46.jsonl`
- SHA-256: `cdeef4e1499a2a57c362029cda43cb2f062a726ec4b2899c25d122d4b87af53b`
- Source modified: `2026-06-21T13:59:12+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf\_635348d6-51c`
- session_id: `9f6357d8-0e35-491a-a830-052e094c7f37`

Transcript

1. user (2026-06-21T13:56:27.257Z)

Synthesize a clear answer to: "where does the work making the tool Cloud-Run-friendly lie, and is it already being worked on?" for the engine repo.

DOCS/ROADMAP: {"adr":"Cloud Run + GPU as the serving model was ratified across a 4-ADR chain in C:/Users/avidu/Projects/badminton-highlight-indexer/docs/archives/decisions/DECISIONS.md:\n\nADR-011 (2026-06-20, RATIFIED) ? overrode the prior \"rent nothing / not a service\" posture and DEFERRED M3 (serving endpoint + per-video billing) + M4 (scale-to-zero + cost guard). Carries the owner-gated \$06 gates: DPA/consent for non-owner uploads, collector md5 keying, WASB-C3 (weights provenance).\n\nADR-012 (2026-06-20, RATIFIED, DECISIONS.md:598) ? the substantive serving ratification: \"Cloud Run + GPU (L4), scale-to-zero, is the supported SERVING deployment model\" for pay-per-video economics (per-request GPU-seconds = the per-video bill, zero idle cost, co-located with gs://khelsutra-rally-corpus). Stay GPU-native (L4/T4); TPU deferred; NVDEC named as the decode lever. First work item = containerize python -m backend.worker (calls out the WASB torch-1.11 conda env as the known hard part). Edge auth named a hard prerequisite before public serving.\n\nADR-013 (2026-06-20, RATIFIED, DECISIONS.md:661) ? GCP is the SOLE compute stack (training + serving); DigitalOcean dropped for compute; provision via deploy/gcp/only.\n\nADR-014 (2026-06-21, RATIFIED ? M0 design sign-off, build M1+, DECISIONS.md:686) ? \"Compute-decoupled rally serving\": one pure 3-stage core (DETECT/WINDOW/STITCH) never branched on profile; ComputeBackend (where) + StorageBackend (what-bytes) selected by one startup config; ships truly-local + cloud-serving + cpu-mock; stitch runs on Cloud Run so its cost is metered. REFINES (does not supersede) ADR-011/012 by realizing their deferred M3/M4; builds on ADR-013.\n\nOwner gates (per the \$7 tracker): M5 owner real-GPU; M6/M7 owner GCP spend (within the D17 \$100/?10k cap, delete-after); M8 owner real prices/billing; M9 owner privacy/counsel (DPA/consent + edge auth); M10/M11/M12 owner-gated (BYO approval, consent/privacy, OAuth/consent). The 5 originally-gating open questions are locked as D7?D12; the smaller picks as D16; the verification budget as D17.\""},\"cloudRunDesign\":\"SOURCE OF TRUTH: docs/COMPUTE_DECOUPLED_SERVING/README.md \$4 + \$2 decisions, plus deploy/cloudrun/{Dockerfile,README.md}.\n\nJOBS vs SERVICE: Cloud Run JOBS, NOT Services-with-GPU (D16). One Job execution runs the detect+stitch unit. Rationale: it fits the async upload?poll?reel flow + scale-to-zero, and reuses the existing async job control plane verbatim (JobWaiting / claim_due_job / ExecutionPolicy WAIT_AND_RESUME). Implemented as backend/compute/CloudRunCompute (backend/compute/cloud_run.py): get_compute_backend(config) returns a backend ONLY when compute_target=cloud_run (else None ? in-process path, default-OFF).\n\nGPU vs CPU: GPU-native, NVIDIA L4 (cuda), detector_impl=native (the Linux-native WASB runner). DETECT runs on GPU. WINDOW is a CPU-pure function (trajectory_to_action_windows). STITCH is real CPU/ffmpeg work and is CO-LOCATED in the SAME Cloud Run GPU job (D4/D16) so gpu_active_seconds (detect) + wall_seconds (detect+stitch) + bytes_out (reel egress) all land on ONE metered invocation ? the per-job cost ledger. Stitch-on-laptop would hide that cost. Codec (D16): libx264 ENCODE (protects the D5 byte-determinism / parity guarantee) + NVDEC for DECODE only (speed lever, never touches output bytes); NVENC encode is DEFERRED behind a config knob. cpu-mock profile (detector_impl=stub) runs the whole thing GPU-free for \$0 CI.\n\nSCALE-TO-ZERO: D7 ? accept the ~1?3 min cold-start (amortized over a multi-minute async job); NO warm pool (a ~\$500/mo idle pool would break per-video billing). Revisit a warm lane only on a UX complaint.\n\nDISPATCH + COMPLETION DETECTION: The control plane (DEPLOYMENT_PROFILE=cloud-

serving ? compute_target=cloud_run) dispatches a Cloud Run Job execution (injectable client), then BUCKET-POLLS a done-marker for completion ? NOT the Cloud Run API status. The marker is <out_uri>/_dispatch/<token>.done, polled via execution_policy.poll_until + the storage layer, so it reuses JobWaiting/claim_due_job and survives a control-plane restart (D16). The dispatched container runs INLINE ? CloudRunCompute forces compute_target=local_gpu + indexing.detector_impl=native in the per-execution args so the Job never re-dispatches itself; the worker writes the --done-uri marker on success. Known gap (follow-up): a FAILED job currently writes NO marker, so the dispatcher's bounded poll times out (ExecutionPolicy.max_wait_sec ? clean rc 4); a failure-marker carrying the real non-zero rc (to distinguish failed from hung/cold-start-slow) is deferred. Reels returned via signed URL. Input cap: hard-reject <2GB for v1 (a config cap, raise to 4GB later in one line, matching the M2 chunked-upload threshold).

CONTAINERIZATION + THE WASB conda-env CONCERN:

deploy/cloudrun/Dockerfile builds the L4 image FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04, adds ffmpeg (stitch/decode) + DejaVu fonts (watermark filter). It deliberately carries TWO separate python stacks to avoid a framework conflict (same split as the native GPU box, deploy/digitalocean/gpu_setup.sh): (1) the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching; (2) a conda 'wasb' env (python 3.8 + torch 1.11.0+cu113 / torchvision 0.12.0+cu113, from nttcom/WASB-SBDT) runs the WASB shuttle detector as a SUBPROCESS ? the native runner shells out to \$WASB_PYTHON so the app's torch never co-installs with WASB's torch-1.11. ENV wires WASB_PYTHON=/opt/conda/envs/wasb/bin/python, WASB_REPO_DIR, WASB_WEIGHTS, RALLY_DETECTOR_IMPL=native; ENTRYPOINT is python3 -m backend.worker. This dual-runtime packaging is exactly the "known hard part" ADR-012 flagged. WASB-C3 concern: the pretrained badminton weights are academic-data-trained, provenance NOT cleared for commercial sharing ? they are NOT baked into the image; they are STAGED AT RUNTIME (bucket fetch or mount) with \$WASB_WEIGHTS pointed at them. WASB-C3 must be resolved before any public, non-owner serving. CI does NOT build this image (a CUDA+conda image is too heavy); the dispatch shim is proven with a fake client (tests/test_compute_backend.py). The real build/push/L4 run is the M7 owner step under the D17 budget.

trackerFile": "C:/Users/avidu/Projects/badminton-highlight-indexer/docs/COMPUTE_DECOUPLED_SERVING/README.md ? \$7 \"Deliverables & progress tracker\" is the explicit source of truth (legend: ? Todo ? In progress ? Done ? Gated). The \$11 Changelog gives the per-milestone landing notes, and \$2 (Decisions D1?D17) + \$4 (Design) back it.

Supporting: deploy/cloudrun/{Dockerfile,README.md}, docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_truly-local_TEMPLATE.md, and ADR-011/012/013/014 in docs/archives/decisions/DECISIONS.md.

id	title	cloudRunRelevant	status
"M0"	"Design doc + project dir + runlogs/ + ADR-014"	false	"? Done (? APPROVED 2026-06-21, owner sign-off)"
"M1"	"deployment.profile + compute_target config + preset bundles + precedence/auto-detect (no behaviour change, profile='default'"	true	"? Done (PR #279)"
"M2"	"input_backend/input_root; wire main CLI/API input through StorageRef.parse_uri (local=identity)"	true	"? Done (PR #280)"
"M3"	"Configurable output/scratch base ? kill hardcoded output/ (default stays output/, byte-identical)"	true	"? Done (PR #282) ? was ? default-OFF + Launch Report on flip; landed default-OFF so no Launch Report needed"
"M4"	"DeviceContext + WASB CPU-fallback (device='auto'); unified healthcheck/device wiring"	true	"? Done (PR #283) ? safe, cuda default unchanged"
"M5"	"truly-local profile end-to-end (preset + startup checks) + first committed runlog"	true	"? In progress (PR #286: preset+startup-checks+L4 parity test ?; real-GPU truly-local runlog ? owner GPU)"
"M6"	"Cloud Run GPU dispatch shim (control plane ? job via storage ? re-claim) + containerized worker image (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests"	true	"? In progress (PR #287: shim+image+mocked tests ?; real build/run = M7 ? owner GCP spend)"
"M7"	"cloud-serving profile e2e on L4: upload<2GB + gdrive/bucket ? detect+stitch on Cloud Run ? reel via signed URL; DEPLOY_REPORT"	true	"? Planned / ? Gated (owner spend, D17 \$100/?10k budget)"
"M8"	"Per-job cost ledger (v6 migration) + pricebook + aggregation + /api/.../cost"	true	"? In progress (PR #288: ledger+pricebook+/cost ? default-OFF \$0; real GCP prices + worker usage-emission ? owner/follow-up)"
"M9"	"Edge auth (Cf-Access extraction) + per-tenant scoping on user_id; DPA/consent gate wiring"	true	"? Planned / ? Gated (owner privacy/counsel)"
"M10"	"byo_worker / zero-infra-BYO ? design-only, DEFERRED"	true	"? Planned / ? Gated (explicit owner approval; design-only, deferred)"
"M11"	"Data-flywheel harness (D12): capture upload + Gemini reel/rally output ? reliably store ? shadow-eval owned-vs-Gemini ? promote to golden TRAINING set"	false	"? Planned / ? Gated (owner consent/privacy)"
"M12"	"gdrive-api (OAuth) StorageBackend (D14 Northstar): read		

source from + write reel back to user's Google Drive next to source", "cloudRunRelevant": true, "status": "? Planned / ? Gated (owner OAuth/consent)"}], "notes": "SCOPE OF \"Cloud-Run-friendly\": Milestones flagged cloudRunRelevant cover the chain that makes the tool deployable on Cloud Run with per-video billing ? config/profile layer (M1), URI-aware input (M2), de-hardcoded output base needed for bucket workspaces (M3), device portability incl. CPU fallback (M4), the truly-local twin profile + parity test that proves the core is profile-invariant (M5), the actual Cloud Run dispatch shim + container image (M6), the live L4 e2e (M7), per-job cost ledger / per-video billing (M8), edge auth/tenancy (M9), BYO compute (M10), and the OAuth Drive round-trip storage backend (M12). M11 (data-flywheel) is a quality/training-loop harness riding on cloud jobs, not Cloud-Run plumbing itself, so it is flagged false.\n\nSTATUS SUMMARY as the tracker marks it: M0?M4 ? Done (the config/input/output/device foundation is COMPLETE, all merged #279/#280/#282/#283). M5/M6/M8 ? In progress ? the default-OFF CODE has landed (#286/#287/#288) but each retains an owner gate for the real run/spend/calibration. M7/M9/M10/M11/M12 ? Planned, all ? owner-gated. Per the 2026-06-21 changelog, the owner gave a blanket go to WRITE+MERGE the default-OFF code for every remaining gated milestone (one isolated PR each), reserving the gate only for the real run/spend/counsel/calibration ? which is why M5/M6/M8 read as code-done-but-?.\n\nCURRENT BRANCH note: the working branch is feat-serving-m4-device-context; HEAD (44c9f01) is the M4 (#283) tracker/changelog commit. The README on disk already reflects M5?M8 PRs (#286?#288) landing, so the doc is ahead of this branch's git history ? read the README \$7 tracker + \$11 changelog as authoritative, not the local branch's commit log. NOTE: `git pull --ff-only` FAILED at session start (\\"Cannot fast-forward to multiple branches\\") ? I did NOT force anything (per global convention) and read local state only; the local checkout may lag remote.\n\nKEY HONEST CAVEATS (README \$6): green CI proves plumbing + window/stitch determinism + profile-parity for \$0 but NOT field quality or that real-GPU detection is good; cross-GPU detection is NOT byte-identical (parity is a window-level claim); cloud-serving ships with Gemini handoff ON (D11) until the owned model clears the #172 floor; WASB-C3 (weight sharing) + DPA/consent remain owner/counsel-gated before public non-owner serving."}

CODE: {"computeRegistry": "backend/compute/base.py defines the seam as an ABC + dataclasses, NOT a Registry-singleton (unlike segmenters/storage). ComputeBackend(ABC) has a single abstractmethod run_infer(self, spec: ComputeJobSpec) -> ComputeResult (base.py:46-53); ComputeJobSpec frozen dataclass (base.py:20-30) carries video_uri, out_uri, optional segmenter, config_overrides (a tuple so the spec stays frozen/hashable); ComputeResult frozen dataclass (base.py:33-43) carries returncode (0 ok / 2 validation-config / 3 segmenter-fail / 4 = remote dispatch timeout or empty/unreadable marker), outputs_uri, video_id, report. The selection logic lives in backend/compute/__init__.py as a lazy FACTORY get_compute_backend(config, *, run_client=None, policy=DEFAULT_POLICY, token=None) -> Optional[ComputeBackend] (__init__.py:32-61): it returns None for any compute_target other than \"cloud_run\" (__init__.py:47-49) and only lazily imports + builds CloudRunCompute for \"cloud_run\" (__init__.py:51-61). So there is NO ComputeBackend Registry singleton with @register decorators ? it is a single-branch factory function modeled on backend/storage's lazy factory idiom (base.py:3-10, __init__.py:1-7).\", \"cloudRunBackend\": \"backend/compute/cloud_run.py implements CloudRunCompute(ComputeBackend) (cloud_run.py:82-156). DISPATCH/SUBMIT: run_infer (cloud_run.py:103-132) builds worker argv [\"infer\\\", \"--video-uri\\\", ..., \"--out-uri\\\", ..., \"--done-uri\\\", f\"{out_root}/_dispatch/{token}.done\\\"] (token = injected self._token or fresh uuid4 per call, cloud_run.py:105-108), appends --segmenter and every --set k=v from spec.config_overrides PLUS _DEFAULT_CONTAINER_OVERRIDES = (\"deployment.compute_target=local_gpu\\\", \"indexing.detector_impl=native\\\") (cloud_run.py:41-44, 109-112) so the dispatched container runs INLINE and never re-dispatches, then calls self._client.run_job(job_name, argv, region, project) (cloud_run.py:114-115). POLL/AWAIT: marker = poll_until(lambda: self._read_marker(done_uri), self._policy, label=\\\"cloud-run-infer\\\") (cloud_run.py:119-120) using execution_policy.poll_until (execution_policy.py:125-151; default poll_every_n_sec=10s, max_wait_sec=1800s, raises PolicyTimeout on exhaustion). COMPLETION DETECTION IS A BUCKET-POLL of a storage marker, NOT the Cloud Run API status: _read_marker (cloud_run.py:135-140) parse_uri(done_uri) then get_storage(ref.backend).exists(ref), returning None (keep polling) until the worker writes the marker; once present it reads the JSON (_read_json, cloud_run.py:142-155, downloads to a temp local dst first since gcs/s3 require an explicit dst). Then it extracts video_id + returncode (a MISSING returncode defaults to FAILURE 1, never 0; an empty/unreadable marker is treated as failure, cloud_run.py:121-128), composes outputs_uri = f\"{out_root}/outputs/{video_id}/\", reads report.json, returns ComputeResult (cloud_run.py:129-132). The marker is written by the worker via _write_done_marker (backend/worker/__main__.py:140-153, {video_id, returncode, segment_count, status} pushed with storage.put), which CloudRunCompute consumes ? so the completion signal is storage-backed (D16), reusing the existing park/resume primitives.\"\", \"wiring\": \"The compute seam is wired into the

STANDALONE CLI worker ONLY, NOT into /api/process. backend/worker/__main__.py::cmd_infer (worker/__main__.py:156-168) calls compute = get_compute_backend(config) and, if non-None, returns _dispatch_remote(compute, args) (worker/__main__.py:116-137), which builds a ComputeJobSpec from argv and calls compute.run_infer(spec), mapping PolicyTimeout -> rc 4 (worker/__main__.py:124-137); if None it falls through to the unchanged inline PULL->hash->validate->segment path (worker/__main__.py:170+). This worker is `python -m backend.worker infer ...` ? the same binary the Cloud Run container runs. CRITICAL GAP: the HTTP request/job flow does NOT use this seam. POST /api/process (backend/main.py:268-308) only fast-fails + creates a 'queued' jobs-table row and calls _get_executor().submit(_drain_due_jobs) (main.py:304). _drain_due_jobs (backend/api/job_worker.py:132-153) -> _run_claimed_job (job_worker.py:72-98) -> backend.main._execute_processing (main.py:148+) runs the pipeline IN-PROCESS on the ThreadPoolExecutor (job_worker.py:30-38, max_workers=1). _execute_processing NEVER calls get_compute_backend / compute.run_infer ? it always hashes/validates/segments locally in the API process. So today every /api/process job runs in-process; the Cloud Run dispatch path is reachable only by invoking the backend.worker CLI directly with a cloud_run profile. No dispatcher bridges the API job worker to the ComputeBackend yet.", "flagsDefault": "Selection is config.deployment.compute_target (Literal[\"\", \"local_gpu\", \"cloud_run\", \"cpu_mock\"], DEFAULT \"\" ? backend/config/models.py:351). get_compute_backend returns non-None ONLY for compute_target == \"cloud_run\" (__init__.py:47-49); \"\", \"local_gpu\", \"cpu_mock\" all return None => in-process/inline. compute_target is normally set via deployment.profile (Literal[\"\", \"truly-local\", \"cloud-serving\", \"cpu-mock\"], DEFAULT \"\" ? models.py:348), whose presets (backend/config/presets.py:43-63; COMPUTE_TARGET_FOR_PROFILE presets.py:32-36) map cloud-serving->cloud_run, truly-local->local_gpu, cpu-mock->cpu_mock; an explicitly-blanked compute_target under an active profile is back-filled to the profile's canonical target by the loader (backend/config/loader.py:167-181). CONFIRMED DEFAULT-OFF: with no profile selected both profile and compute_target are \"\" (models.py:348,351; presets.py:11-16), so get_compute_backend returns None and the worker runs inline ? Cloud Run is default-OFF and in-process is the default. Cloud Run coordinates come from ENV not config: CLOUD_RUN_JOB (default \"rally-worker\"), CLOUD_RUN_REGION (default \"asia-south1\"), GOOGLE_CLOUD_PROJECT (__init__.py:54-57). Related M8 BillingConfig.enabled is also default-OFF (models.py:367).\", \"mockedVsReal\": \"REAL: CloudRunCompute orchestration (argv build, container-override injection, bucket-poll loop, marker parsing with fail-safe returncode, report fetch) is real and unit-testable; storage layer (parse_uri/get_storage/fetch/put/exists) is real; the worker's _write_done_marker (worker/__main__.py:140-153) is real; config/profile/loader plumbing is real. The real client GoogleCloudRunJobClient.run_job (cloud_run.py:60-79) DOES contain real Cloud Run API calls when reached ? it lazy-imports google.cloud.run_v2, builds JobsClient().job_path(project, region, job_name), wraps argv in RunJobRequest.Overrides.ContainerOverride(args=argv), and calls client.run_job(RunJobRequest(...)) ? BUT it is marked '# pragma: no cover - exercised only on the real M7 run' and google-cloud-run is intentionally NOT a CI/test dependency (cloud_run.py:60-72), so this path has never actually run; owner-verification is deferred to M7 (__init__.py:45, cloud_run.py:18-19). MOCKED IN PRACTICE: the trigger is an injected abstraction CloudRunJobClient (ABC, cloud_run.py:47-57); tests pass a FAKE client (no GCP) and that is the ONLY exercised path. A real cloud dispatch still needs: (1) install google-cloud-run + valid GOOGLE_CLOUD_PROJECT/region creds on the control plane; (2) a deployed Cloud Run GPU Job named per CLOUD_RUN_JOB whose container runs `python -m backend.worker infer`; (3) a real gcs storage backend reachable by both dispatcher and container for the gs:// video/out/done URIs; (4) owner-verification that ContainerOverride(args=argv) truly overrides job args and the container writes the done-marker; (5) crucially, an API-side dispatcher to route /api/process jobs through the ComputeBackend (bypassed today); (6) M8 usage-emission hooks (cost stays 0 without them, job_worker.py:101-129).\", \"gaps\": [\"WIRING GAP: get_compute_backend is consulted ONLY in backend/worker/__main__.py::cmd_infer (worker/__main__.py:166); the HTTP path POST /api/process -> _drain_due_jobs -> _run_claimed_job -> backend.main._execute_processing (main.py:148, job_worker.py:86) NEVER calls it, so all API jobs run in-process on the ThreadPoolExecutor (job_worker.py:30-38). Cloud Run dispatch is reachable only by invoking `python -m backend.worker infer` directly with a cloud_run config; no dispatcher bridges the async job worker to the ComputeBackend.\", \"REAL CLOUD CALLS UNVERIFIED: GoogleCloudRunJobClient.run_job (cloud_run.py:60-79) is the only real-API code but is pragma-no-cover and google-cloud-run is intentionally excluded from test deps; the real dispatch has never been exercised (owner-gated to M7). CI only runs the injected fake client.\", \"RETURN-PATH ASYMMETRY: ComputeResult carries returncode + outputs_uri + report, but _dispatch_remote (worker/__main__.py:116-137) only propagates result.returncode as the process exit code; outputs_uri/report are logged, not surfaced; there is no API consumer of ComputeResult yet.\", \"COMPLETION SIGNAL IS A STORAGE MARKER, NOT CLOUD RUN STATUS (by design, D16): a container that dies BEFORE writing the marker

is detected only by poll_until exhausting max_wait_sec=1800s -> PolicyTimeout -> rc 4 (cloud_run.py:119-120, worker/__main__.py:124-134); the Cloud Run execution's own failure status is never consulted, so a fast-failing job still costs the full ~30min poll budget before being reported failed.", "COST METERING ABSENT: even with billing.enabled + a real pricebook, recorded cost is 0 because the pipeline does not emit result['usage'] yet (job_worker.py:101-129, models.py:363-365).", "compute_target='cpu_mock' selects NO dedicated backend: get_compute_backend treats cpu_mock the same as '/local_gpu (returns None, __init__.py:49); the cpu-mock profile relies on indexing.detector_impl=stub, not a ComputeBackend implementation."]}

DEPLOY/CONTAINER: {"dockerfile": "deploy/cloudrun/Dockerfile (C:/Users/avidu/Projects/badminton-highlight-indexer/deploy/cloudrun/Dockerfile, 62 lines) builds a single image for the rally GPU **Job** worker (COMPUTE_DECOUPLED_SERVING M6/M7, ADR-012/D16).\\n\\nBASE IMAGE: `nvidia/cuda:12.4.1-runtime-ubuntu22.04` (CUDA 12.4 runtime, Ubuntu 22.04) ? a GPU/CUDA base, not a slim/CPU base.\\n\\nOS DEPS: apt-installs `ffmpeg` (decode/stitch), `fonts-dejavu-core` (watermark filter), `git`/`curl`/`ca-certificates`/`build-essential`, and the distro `python3`/`python3-pip`/`python3-venv`.\\n\\nTWO DELIBERATELY-SEPARATE PYTHON STACKS (same split as the native GPU box deploy/digitalocean/gpu_setup.sh, to avoid a framework/torch conflict):\\n 1) MAIN APP ENV ? system python3; `pip install .` of this repo from pyproject.toml (COPY pyproject.toml, README.md, backend/, training/). Drives the pipeline + ffmpeg stitching. Notably the main env installs NO torch (detection is delegated out to the WASB subprocess).\\n 2) NATIVE WASB CONDA ENV ? installs Miniconda to /opt/conda, accepts conda TOS for main+r channels, creates conda env `wasb` with python=3.8, clones https://github.com/nttcom/WASB-SBDT.git into /opt/models/WASB-SBDT, and pip-installs torch==1.11.0+cu113 / torchvision==0.12.0+cu113 (CUDA 11.3 wheels, via the pytorch cu113 extra-index) plus hydra-core, omegaconf, pandas, opencv-python-headless, pillow, scipy, tqdm, pyyaml, matplotlib, numpy<1.24, and the WASB repo's own requirements.txt if present. The native runner shells out to \$WASB_PYTHON as a subprocess, so the app's torch never co-installs with the detector's torch-1.11 stack.\\n\\nWASB WEIGHTS: NOT baked in (WASB-C3 ? pretrained badminton weights are academic-data-trained, provenance not cleared for commercial sharing). The image only sets \$WASB_WEIGHTS to a default path (/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar); weights are expected to be staged at runtime (bucket fetch or mount) and pointed at via \$WASB_WEIGHTS.\\n\\nENV WIRING: WASB_PYTHON=/opt/conda/envs/wasb/bin/python, WASB_REPO_DIR=/opt/models/WASB-SBDT, WASB_WEIGHTS=<default path>, RALLY_DETECTOR_IMPL=native (so the container uses the Linux-native WASB runner, not the WSL/cuda path). The native runner finds the env+repo purely by these env vars (no `conda activate`).\\n\\nENTRYPOINT: `[\"python3\", \"-m\", \"backend.worker\"]` ? i.e. the WORKER, not the FastAPI server. The dispatcher supplies per-execution args at run time: `infer --video-uri gs://? --out-uri gs://? --done-uri gs://? --segmenter wasb\_hybrid --set ?`. backend/worker/__main__.py cmd_infer runs detect+window+stitch INLINE (co-located so CPU/wall/egress meter into one job, D4), then writes the report and (when --done-uri is passed) a tiny JSON completion marker {video_id, returncode, ...} to the bucket.", "buildable": "SCAFFOLD / owner-built-once, NOT built by CI and not part of the automated test loop. Findings: (1) NO CI builds it ? the only GitHub Actions workflow is .github/workflows/ci.yml, which runs the mocked offline test suite on a CPU runner (it even installs CPU-only torch and explicitly notes torch is not a pyproject dep); a repo-wide grep for `gcloud builds submit`/`gcloud run`/`buildx`/`docker build` finds zero hits in CI. The Dockerfile header states plainly: `\"CI does NOT build this (a CUDA image is too heavy)\"`. (2) NO build/push/deploy SCRIPT exists ? there is no shell script that builds or pushes this image. The deploy/gcp/*.sh scripts (provision.sh, create_gpu_vm.sh, ops-agent-startup.sh, teardown.sh) only provision a GPU VM + GCS bucket + IAM + budget; none reference Cloud Run, the Dockerfile, or `gcloud run`/`gcloud builds`. The only build/push/deploy instructions are COPY-PASTE commands documented in deploy/cloudrun/README.md (gcloud builds submit; gcloud run jobs create), to be run by the owner. (3) The control-plane dispatch shim IS real + landed + mock-tested (backend/compute/cloud_run.py CloudRunCompute, tests/test_compute_backend.py with a fake CloudRunJobClient ? the real GoogleCloudRunJobClient lazy-imports google-cloud-run, which is intentionally NOT a test dependency, so that path is never exercised in CI). Per the README/Dockerfile, the real build+push+run on an L4 is the **M7 owner step** within a \$100/?10k budget; status in deploy/cloudrun/README.md is `\"dispatch shim + image are landed + mock-tested (M6); the real build + push + run on an L4 is the M7 owner step.\"` So: the image is intended to be buildable and the worker/dispatch logic is exercised by tests, but the image itself is verified only by a manual owner run, not by any automated build/push/deploy/CI.", "gpuStory": "The image is GPU-FIRST and Cloud-Run-Job-only. Base is the CUDA 12.4 runtime and the WASB conda env pins torch 1.11.0+cu113 / torchvision 0.12.0+cu113 (CUDA wheels), so the detector subprocess expects an NVIDIA GPU. The Cloud Run resource it targets is a GPU **Job** (not a Service): deploy/cloudrun/README.md \$2 creates it with `gcloud run jobs create rally-worker --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi --max-retries 0 --task-timeout 3600`, scale-to-zero (D7, no

warm pool); the control plane bucket-polls a done-marker rather than the Cloud Run API. CPU-vs-GPU at the IMAGE/deploy level is NOT a configurable axis here ? there is no CPU variant of this image, no `--gpu`-optional path, and ffmpeg is NVDEC-decode + libx264-encode (the image keeps ffmpeg's default libx264 for parity, D16). A CPU FALLBACK does exist but only at the CODE level, not in the container build: the current branch's M4 work (feat/serving-m4-device-context, commit 6932d4a) added a DeviceContext (backend/pipeline/detectors/device.py) and native WASB CPU-fallback where device="auto" can degrade to CPU while device="cuda" stays strict. That fallback is a property of the native runner / config, not something this Dockerfile's cu113 torch build or the L4 Job spec is set up to exploit ? the cu113 wheels would still run on CPU only via torch's CPU code paths if no GPU were present, but the documented serving path always requests an L4. In short: native WASB packaging into a CUDA container is feasible and is exactly what this Dockerfile does (conda py3.8 + torch-1.11+cu113 mirroring the verified DO/native GPU box); the open caveat is weights provenance (WASB-C3), not GPU packaging.", "deployScripts": "NONE beyond the Dockerfile + manual README commands. deploy/cloudrun/ contains only Dockerfile + README.md ? no build.sh/deploy.sh/cloudbuild.yaml. The actual Cloud Run lifecycle lives as documented copy-paste commands in deploy/cloudrun/README.md: \$1 `gcloud artifacts repositories create ? && gcloud builds submit --tag ? :latest -f deploy/cloudrun/Dockerfile .`; \$2 `gcloud run jobs create rally-worker --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi --max-retries 0 --task-timeout 3600 --set-env-vars WASB\_WEIGHTS=?`; \$3 wiring the dispatcher via env (DEPLOYMENT_PROFILE=cloud-serving ? compute_target=cloud_run, CLOUD_RUN_JOB / CLOUD_RUN_REGION / GOOGLE_CLOUD_PROJECT). The deploy/gcp/ scripts are unrelated to Cloud Run ? provision.sh (GCS bucket + rally-worker SA + Storage Object Admin + Ops-Agent writer roles + \$100 budget + GPU-quota check), create_gpu_vm.sh (spins up a g2-standard-4 L4 / n1+t4 VM with the Ops Agent ? a plain GPU VM, NOT Cloud Run), ops-agent-startup.sh, teardown.sh (defaults to delete). There is no `gcloud run deploy` or `gcloud run jobs` anywhere in deploy/gcp/ (grep-confirmed). M7 acceptance is a manual owner runlog (DEPLOY_REPORT_cloud-serving_<date>.md under docs/COMPUTE_DECOUPLED_SERVING/runlogs/), not an automated deploy.", "gaps": ["No CI builds, lints, or smoke-tests the image ? the CUDA+conda image is deemed too heavy for CI (ci.yml is CPU-only mocked tests), so image rot can go undetected until the owner's manual M7 build.", "No build/push/deploy script ? only copy-paste gcloud commands in deploy/cloudrun/README.md; nothing automates `gcloud builds submit` or `gcloud run jobs create`, and deploy/gcp/ has zero Cloud Run wiring.", "WASB weights are not baked in (WASB-C3 provenance not cleared for commercial sharing) and there is NO staging step in the image or scripts ? the README assumes a runtime bucket-fetch/mount that is not yet automated; serving cannot actually run until weights are staged and WASB-C3 is resolved.", "The real Cloud Run client (GoogleCloudRunJobClient) and the actual L4 build/run are owner-verified only (M7), never exercised by tests ? google-cloud-run is intentionally excluded from test deps, so that whole real path is untested in CI.", "Known correctness gap documented in the README: a FAILED Job writes no done-marker, so the dispatcher's bounded bucket-poll just times out to a generic rc 4 and cannot distinguish a real failure from a slow cold-start; a failure-marker carrying the real rc is a noted follow-up.", "No CPU variant / GPU-optional image: the image is hard-pinned to CUDA (cu113 torch) and the Job spec always requests an L4; the M4 DeviceContext CPU-fallback exists only in code (backend/pipeline/detectors/device.py + native_wasb_runner.py), not as a buildable CPU deploy target, so there is no documented way to serve on Cloud Run without a GPU.", "WORKDIR/COPY caching: `pip install .` runs after COPYING backend/ + training/, so any source change re-runs the heavy main-env install; the expensive conda/WASB layer is earlier so it caches, but there is no requirements-only pre-copy step for the app env."]]

ACTIVITY: {"mergedPRs":["#288 feat(serving): M8 ? per-job cost ledger + versioned pricebook + /cost API (default-OFF)", "#287 feat(serving): M6 ? Cloud Run GPU dispatch shim + worker image (default-OFF, mocked)", "#286 feat(serving): M5 ? truly-local profile per-profile startup checks + L4 profile-parity test (default-OFF)", "#285 docs(serving): resolve S8 design picks (D16/D17) + reconcile per-video P3 vs M3", "#284 test(api): lock gs://|gcs:// /api/process -> queryable API-DB video_id (B-P1 guard)", "#283 feat(serving): M4 ? DeviceContext + native WASB CPU-fallback (device=auto, cuda strict)", "#282 feat(serving): M3 ? configurable output/scratch base (de-hardcode output/, default-OFF)", "#281 fix(server): drain async jobs under `python -m backend.main` (double-module hang)", "#280 feat(serving): M2 ? URI-aware input (input_backend/input_root, local=identity)", "#279 feat(serving): M1 ? deployment.profile + compute_target config (pure, additive, default-OFF)", "#278 docs(serving): M0 APPROVED + lock D13-D15 (switchable profiles, Drive Northstar, consent-not-surprise)", "#277 docs(serving): lock the 5 M0 gating decisions (D7-D12) + data-flywheel harness (M11)", "#289 fix(api): resolve a bucket-ingested video's source to a local file before stitching (compile path)", "#268 feat(api): GET /api/health + return download_url from /api/compile (M1/P6)", "#263 docs(adr): ADR-012 ? GPU-native compute; TPU deferred; Cloud Run+GPU is the serving model"], "openPRs":["#290 feat(serving): M9-auth ? Cloudflare Access edge-auth + per-tenant owner_id (default-OFF) ?

opened 2026-06-21T13:49Z, not draft, +406/-13"], "activelyWorked": true, "whoAndCadence": "Extremely active, concentrated in a single burst on 2026-06-21 (author: Avi Dullu / current session). The serving track went from M0 approval to M8 code merged in roughly half a day: #278 (M0 APPROVED) at 06:59Z, #279 (M1) 07:34Z, #280 (M2) 08:15Z, #281 (fix) 08:22Z, #282 (M3) 08:39Z, #283 (M4) 09:12Z, #284 (test) 09:35Z, #285 (design picks) 09:58Z, #286 (M5) 10:27Z, #287 (M6) 10:53Z, #288 (M8) 11:16Z, #289 (compile fix) 13:29Z ? then #290 (M9-auth) opened 13:49Z and still OPEN. That is ~13 serving PRs in ~7 hours, on top of the earlier decoupled-compute/deploy run (#250-#268, GCP L4 deploy report) over 2026-06-20. Recent origin/master HEAD = e68b583 (#289). The work is one isolated default-OFF PR per milestone, off origin/master, no stacking.", "nextPlanned": "Per the live tracker docs/COMPUTE_DECOUPLED_SERVING/README.md §7: code for M1-M8 + M5/M6 plumbing is merged default-OFF, and M9-auth (#290) is the open PR. The next planned Cloud-Run steps are: (1) M7 ? the real cloud-serving e2e on a GCP L4 (upload<2GB / gdrive / bucket -> detect AND stitch co-located on a Cloud Run JOB -> reel via signed URL) with a committed DEPLOY_REPORT, the key owner-gated real-spend/real-build milestone (D17 \$100/?10k budget; M6 already shipped the Cloud Run dispatch shim + deploy/cloudrun Dockerfile/worker image, mocked, awaiting the real build/push/run). (2) finish M8 (owner inserts real GCP prices as a new pricebook version + wire worker usage-emission). (3) M9 consent/DPA gate wiring (counsel-gated) alongside the edge-auth in #290. (4) M11 data-flywheel harness (capture->store->shadow-eval->goldenize consented adult jobs). (5) M12 gdrive-api OAuth StorageBackend (the Northstar Drive round-trip). M10 byo_worker stays design-only/deferred."}]

Produce: a one-paragraph headline; the concrete files/dirs holding the work (each with its role); status by layer (design/code/container/deploy: done vs in-progress vs planned); a decisive verdict on whether it is actively being worked (the concurrent serving session); the genuinely remaining Cloud-Run gaps; and what NOT to duplicate / how to coordinate to avoid colliding with the active session. Return structured JSON.

2. assistant (2026-06-21T13:56:31.005Z)

I have been given a comprehensive research dossier. Let me verify the most load-bearing claims against the actual files on disk before synthesizing, since the context notes the local checkout may lag remote and the README is "ahead" of this branch's git history.

3. user (2026-06-21T13:56:34.021Z)

```
feat/gcp-ops-agent-and-dryrun-guide
c3dbd00 feat(gcp): always-install Ops Agent on GPU VMs + a cloud-GPU dry-run guide
e68b583 fix(api): resolve a bucket-ingested video's source to a local file before stitching
(#289)
43eccc1 feat(serving): M8 ? per-job cost ledger + versioned pricebook + /cost API (default-OFF)
(#288)
---REMOTE---
e68b583 fix(api): resolve a bucket-ingested video's source to a local file before stitching
(#289)
43eccc1 feat(serving): M8 ? per-job cost ledger + versioned pricebook + /cost API (default-OFF)
(#288)
8cdf65c feat(serving): M6 ? Cloud Run GPU dispatch shim + worker image (default-OFF, mocked)
(#287)
---STATUS---
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
```

4. user (2026-06-21T13:56:34.688Z)

```
backend/compute/:
total 32
drwxr-xr-x 1 avidu 197609  0 Jun 21 16:20 .
drwxr-xr-x 1 avidu 197609  0 Jun 21 19:20 ..
-rw-r--r-- 1 avidu 197609 2783 Jun 21 16:07 __init__.py
drwxr-xr-x 1 avidu 197609  0 Jun 21 19:19 __pycache__
-rw-r--r-- 1 avidu 197609 2483 Jun 21 16:20 base.py
-rw-r--r-- 1 avidu 197609 8275 Jun 21 16:20 cloud_run.py

deploy/cloudrun/:
```

```
total 8
drwxr-xr-x 1 avidu 197609    0 Jun 21 16:20 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 16:11 ..
-rw-r--r-- 1 avidu 197609 3716 Jun 21 16:11 Dockerfile
-rw-r--r-- 1 avidu 197609 3927 Jun 21 16:20 README.md
```

```
docs/COMPUTE_DECOUPLED_SERVING/:
total 88
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:20 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:25 ..
-rw-r--r-- 1 avidu 197609 33780 Jun 21 19:20 README.md
-rw-r--r-- 1 avidu 197609  8643 Jun 21 08:15 TESTING_STRATEGY.md
-rw-r--r-- 1 avidu 197609 10159 Jun 21 08:17 architecture.svg
drwxr-xr-x 1 avidu 197609    0 Jun 21 15:46 runlogs
```

5. assistant (2026-06-21T13:56:40.752Z)

Important ? the local checkout is NOT stale. `origin/master` HEAD is `e68b583` (#289), local `master` matches, and I am actually on branch `feat/gcp-ops-agent-and-dryrun-guide` (HEAD `c3dbd00`), not the `feat/serving-m4-device-context` branch the dossier assumed. The README on disk (33KB, modified 19:20) reflects M5-M8 landed. Let me verify the wiring gap claim (the most decisive code finding) and the tracker status directly.

6. user (2026-06-21T13:56:41.730Z)

```
1      """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3      COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4      ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
5      ``None``
6      otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
7      identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
8      ``cpu_mock`` nothing
9      changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
10     """
11
12     from __future__ import annotations
13
14     import os
15     from typing import Any, Optional
16
17     from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
18     from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
19
20     __all__ = [
21         "ComputeBackend", "ComputeJobSpec", "ComputeResult", "get_compute_backend",
22     ]
23
24     def _storage_dict(config: Any) -> dict:
25         storage = getattr(config, "storage", None)
26         if storage is None:
27             return {}
28         if hasattr(storage, "model_dump"):
29             return storage.model_dump() # type: ignore[no-any-return]
30         return dict(storage) if isinstance(storage, dict) else {}
31
32     def get_compute_backend(
33         config: Any,
34         *,
35         run_client: Optional[Any] = None,
36         policy: ExecutionPolicy = DEFAULT_POLICY,
```

```

37         token: Optional[str] = None,
38     ) -> Optional[ComputeBackend]:
39         """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
40         in-process (the default-OFF path).
41
42         Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
43         Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
44         / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
45         pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
46         """
47         target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
48         if target != "cloud_run":
49             return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
50
51         # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
52         from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
53
54         job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
55         region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
56         project = os.environ.get("GOOGLE_CLOUD_PROJECT")
57         client = run_client if run_client is not None else GoogleCloudRunJobClient()
58         return CloudRunCompute(
59             run_client=client, job_name=job_name, region=region, project=project,
60             storage_config=storage_dict(config), policy=policy, token=token,
61         )
62

```

7. user (2026-06-21T13:56:42.837Z)

```

tests\test_compute_backend.py:12:from backend.compute import ComputeJobSpec, get_compute_backend
tests\test_compute_backend.py:68:     assert get_compute_backend(_cfg("")) is None
tests\test_compute_backend.py:69:     assert get_compute_backend(_cfg("local_gpu")) is None
tests\test_compute_backend.py:70:     assert get_compute_backend(_cfg("cpu_mock")) is None
tests\test_compute_backend.py:74:     backend = get_compute_backend(_cfg("cloud_run", "gcs"),
run_client=_FakeRunClient())
tests\test_compute_backend.py:198:     monkeypatch.setattr(wmain, "get_compute_backend", lambda
config: _FakeBackend())
backend\worker\_main_.py:28:from backend.compute import ComputeJobSpec, get_compute_backend
backend\worker\_main_.py:116:def _dispatch_remote(compute: Any, args: argparse.Namespace) ->
int:
backend\worker\_main_.py:131:     result = compute.run_infer(spec)
backend\worker\_main_.py:163:     # DEFAULT-OFF: get_compute_backend returns None for
"""/local_gpu/cpu_mock ? fall through to the
backend\worker\_main_.py:166:     compute = get_compute_backend(config)
backend\worker\_main_.py:168:     return _dispatch_remote(compute, args)
deploy\cloudrun\README.md:37: `compute_target=cloud_run`). `get_compute_backend(config)` then
builds `CloudRunCompute` from env:
deploy\cloudrun\README.md:58: (`ExecutionPolicy.max_wait_sec`) and `_dispatch_remote` returns a
clean **rc 4** (not a traceback).
backend\compute\__init__.py:3:COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is
the seam: it returns a
backend\compute\__init__.py:19:     "ComputeBackend", "ComputeJobSpec", "ComputeResult",
"get_compute_backend",
backend\compute\__init__.py:32:def get_compute_backend(
docs\COMPUTE_DECOUPLED_SERVING\README.md:166:[Omitted long matching line]

```

8. assistant (2026-06-21T13:56:46.151Z)

The decisive wiring gap claim is confirmed: `get\_compute\_backend` is consulted only in

`backend/worker/\_\_main\_\_.py` (line 166), never in the API path. Let me confirm the API path does NOT call it, and read the tracker \$7 + changelog for authoritative status.

9. user (2026-06-21T13:56:47.720Z)

```
148: def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
459:     # for a bucket URI (the same seam the process path uses, _execute_processing
main.py:159).
```

10. user (2026-06-21T13:56:49.121Z)

No matches found

11. user (2026-06-21T13:56:49.897Z)

```
1      # Compute-Decoupled Rally Serving ? one pure core, profile-selected compute + storage
2
3      > **Status:** ? **M0 APPROVED** (owner, 2026-06-21) ? **IN PROGRESS (build M1+)** .
**Owner:** Avi . **Created:** 2026-06-21 . **Last updated:** 2026-06-21
4      > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to
`docs/archives/past_projects/` when DONE)
5      > **Tracking anchors:** $7 progress tracker is the source of truth; pointer in memory
`current-state` + `docs/NEXT_STEPS.md` once work starts.
6      > **Relation to existing docs:** the productionization successor to the archived
`DECOUPLED_COMPUTE` (M3/M4); realizes the `ComputeBackend`/`StorageBackend` registries from
[PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) $3; the **truly-local** profile is the
L0 tier of [VIDEO_LOCALITY_MODEL.md](../VIDEO_LOCALITY_MODEL.md) and the `LocalDesktop` backend
of [DESKTOP_APP_PLAN.md](../DESKTOP_APP_PLAN.md).
7      > **Naming:** this was the "Cloud Run + GPU serving subproject" (ADR-012). Renamed
**compute-decoupled serving** because **truly-local is a co-equal, first-class profile**, not an
afterthought.
8      > **Honesty note:** claims tagged `[verified]` (checked against code by the design
workflow `wf_56274ff0`), `[design]` (proposed), `[researched]`.
9
10     ---
11
12     > **? Northstar (D14):** a **mobile, app-like** flow ? point at a **Google Drive** video
? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next
to the source**. Two design promises around it: a user can **switch local?cloud anytime** (D13,
a headline feature), and execution is **never a surprise** ? auto-detect *suggests*, the user
*confirms* (D15).
13
14     ## 0. TL;DR
15     The rally engine is a **pure function of `(input bytes + config) ? outputs`** in three
deterministic stages ? **DETECT ? WINDOW ? STITCH** ? that **no deployment profile may branch
on**. A **`ComputeBackend` (deployment profile)** decides *where* the core runs (a local process
vs a Cloud Run GPU job); a **`StorageBackend`** decides *where bytes come from* (local FS / USB
/ mounted Google Drive / bucket). **One startup config** picks both. We ship **truly-local**
(local GPU + in-process stitch, zero cloud) and **cloud-serving** (upload `<2GB` / gdrive /
bucket ? a Cloud Run GPU job runs detect **and** stitch, so stitch's compute is metered), plus
**cpu-mock** for CI. The single most important caveat: detection is **byte-identical only on the
same GPU** (cross-GPU is sub-pixel-different, per the 2026-06-20 deploy report) ? so the
**parity guarantee is enforced at the WINDOW + STITCH layer**, not detector-CSV bytes.
16
17     ## 1. Problem & goal
18     - **Why now:** DECOUPLED_COMPUTE proved compute *portability* (M0?M2 + M3.5 on a GCP L4)
and is archived; its deferred **M3 (serving endpoint + per-video billing) + M4 (scale-to-zero +
cost guard)** become this project. The owner needs **both** a hosted service *and* a truly-local
mode, with the **core (detect+stitch) unaffected** by which one runs.
19     - **The two user-facing outcomes** (owner, 2026-06-21):
20     - ***(a) Hosted service** ? operate on small **local uploads** (`<2GB`)** *and* load from
**gdrive / a bucket**, spinning up **Cloud Run for both rally segmentation AND stitching**
(stitching is compute-intensive ? run it where its cost is accounted for).
21     - ***(b) Truly-local** ? on a local machine with videos on **local drives/USB + a local
```

GPU**, run inference **and** stitching **completely locally**.

22 - **Good** looks like: the same `video ? rallies ? reel` core gives identical windows + stitched ranges whether it runs on a laptop or Cloud Run; switching is a **config/startup choice**, never a code branch in the core.

23

24 ## 2. Decisions locked

25

#	Decision	Source	Implication
D1	The core is a pure 3-stage function (DETECT/WINDOW/STITCH); no profile branch inside it . design <code>[verified]</code> All profile difference lives in config + the two registries.		
D2	Two registries : <code>ComputeBackend`</code> (where) selected by <code>deployment.profile` + `compute_target`</code> ; <code>StorageBackend`</code> (what-bytes) by <code>input_backend`/`storage.backend`</code> . ADR-014 Realizes PLATFORM_ARCHITECTURE's <code>local\ hosted\ byo_worker`</code> .		
D3	Profiles shipped : <code>truly-local`</code> , <code>cloud-serving`</code> , <code>cpu-mock`</code> (CI); <code>byo_worker`</code> reserved/deferred. ADR-014 Enum/seam reserved so BYO isn't precluded.		
D4	In cloud-serving , STITCH runs on Cloud Run** (co-located with detect) so CPU/wall + egress are metered into the per-job cost ledger. owner (a) Stitch-on-laptop would hide real cost.		
D5	Parity enforced at WINDOW+STITCH (byte-exact for a fixed trajectory); detect is byte-exact only same-GPU. deploy report 2026-06-20 Cross-GPU asserted at the rally/window level, never CSV bytes.		
D6	Default-OFF , smallest-safe-first, one PR per milestone; any core-caller change (e.g. configurable output base) ships behind a flag + a Launch Report. [[launch-report-convention]] Zero-regression discipline.		
D7	Cloud Run cold-start: SCALE-TO-ZERO ? accept the ~1?3 min cold-start (it's an async job: upload?poll?reel, amortized over the multi-minute job). No warm pool (that ~\$500/mo idle would break per-video billing). owner 2026-06-21 M6/M7. Revisit a warm lane only on a UX complaint.		
D8	Pricebook = a versioned DB table; cost computed in USD (matches GCP billing = one source of truth), displayed in INR at a configurable FX rate (Bangalore market); re-versioned when GCP prices change. owner 2026-06-21 M8.		
D9	Edge auth = Cloudflare Access (reuse the site's existing CF). Lock the Cloud Run ingress to the CF proxy + verify the `Cf-Access` JWT signature (so a direct hit to the Cloud Run URL can't spoof the identity header). owner 2026-06-21 M9. One identity system.		
D10	Free-tier "data-for-reels" trade : free reels in exchange for training rights (uploaded footage + derived trajectories/labels); paid tier = no-train (privacy is the paid feature). Carve-outs : train ONLY on attested ADULT footage (minor footage ? reel yes, training pool NO ? DPDP/GDPR need verifiable parental consent); train on DERIVED data + auto-delete raw ; ToS counsel-reviewed ; live training-on-uploads gated to M9 (public signup). Truly-local needs no DPA. owner 2026-06-21 M9 + D12; ties [[paper-coauthor-aim]] / PERSONALIZATION flywheel.		
D11	Quality floor: Gemini handoff ON for cloud-serving until the owned model clears the #172 ship-gate (e.g. ?0.70 multi-court) ? then flip via a Launch Report ; the owned model runs in shadow/eval meanwhile. (Gemini = serving/inference ONLY, never a training source ? CLAUDE.md guardrail.) owner 2026-06-21 M7 + D12.		
D12	Data-flywheel harness is IN SCOPE (owner add, Q5): capture the (consented, adult) uploaded video + the Gemini reel/rally output ? reliably store in the per-video workspace/bucket ? run shadow evals (owned-vs-Gemini, dual-eval-set) ? promote good ones to the golden TRAINING set (human-reviewed labels) so the owned model climbs toward the D11 floor (then Gemini flips off). owner 2026-06-21 new M11 ; ties D10 (consent) + <code>data_pipeline/`</code> + the eval/experiment harness.		
D13	Profile is user-switchable ANYTIME ? a headline FEATURE . The same user runs local today, cloud tomorrow, back to local ? it's just a config/startup choice; the pure core gives equivalent output either way. Advertise it ("your machine *or* our cloud ? your call, switch anytime"). owner 2026-06-21 Honest caveat: equivalent at the rally/window level, not bit-identical across GPUs (D5); a single job runs in one profile, switching is between jobs.		

12. assistant (2026-06-21T13:57:01.399Z)

Confirmed: `backend/main.py::_execute_processing`` (the API job path) has zero references to `get_compute_backend`/`run_infer`/`ComputeBackend``, and `job_worker.py`` has none either. The API

path runs in-process; only the standalone `backend.worker` CLI dispatches to Cloud Run. Now let me read the \$7 tracker and \$11 changelog for the authoritative milestone status.

13. user (2026-06-21T13:57:01.970Z)

```
3:> **Status:** ? **M0 APPROVED** (owner, 2026-06-21) ? **IN PROGRESS (build M1+)** . **Owner:**
Avi . **Created:** 2026-06-21 . **Last updated:** 2026-06-21
6:> **Relation to existing docs:** the productionization successor to the archived
`DECOUPLED_COMPUTE` (M3/M4); realizes the `ComputeBackend`/`StorageBackend` registries from
[PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) $3; the **truly-local** profile is the
L0 tier of [VIDEO_LOCALITY_MODEL.md](../VIDEO_LOCALITY_MODEL.md) and the `LocalDesktop` backend
of [DESKTOP_APP_PLAN.md](../DESKTOP_APP_PLAN.md).
18:- **Why now:** DECOUPLED_COMPUTE proved compute *portability* (M0?M2 + M3.5 on a GCP L4) and
is archived; its deferred **M3 (serving endpoint + per-video billing) + M4 (scale-to-zero + cost
guard)** become this project. The owner needs **both** a hosted service *and* a truly-local
mode, with the **core (detect+stitch) unaffected** by which one runs.
34:| D7 | **Cloud Run cold-start: SCALE-TO-ZERO** ? accept the ~1?3 min cold-start (it's an
async job: upload?poll?reel, amortized over the multi-minute job). **No warm pool** (that
~$500/mo idle would break per-video billing). | owner 2026-06-21 | M6/M7. Revisit a warm lane
only on a UX complaint. |
35:| D8 | **Pricebook = a versioned DB table** ; cost computed in **USD** (matches GCP billing =
one source of truth), **displayed in INR** at a configurable FX rate (Bangalore market); re-
versioned when GCP prices change. | owner 2026-06-21 | M8. |
36:| D9 | **Edge auth = Cloudflare Access** (reuse the site's existing CF). Lock the Cloud Run
**ingress to the CF proxy** + **verify the `Cf-Access` JWT signature** (so a direct hit to the
Cloud Run URL can't spoof the identity header). | owner 2026-06-21 | M9. One identity system. |
37:[Omitted long matching line]
38:| D11 | **Quality floor: Gemini handoff ON** for cloud-serving until the owned model clears
the #172 ship-gate (e.g. ?0.70 multi-court) ? then flip via a **Launch Report** ; the owned model
runs in **shadow/eval** meanwhile. (Gemini = serving/inference ONLY, never a training source ?
CLAUDE.md guardrail.) | owner 2026-06-21 | M7 + D12. |
39:[Omitted long matching line]
41:[Omitted long matching line]
43:[Omitted long matching line]
44:[Omitted long matching line]
55:| **ADR-011** | DECOUPLED scope = M0?M2+M3.5; **deferred M3 (serving) + M4 (billing/cost-
guard)** ; overrode the **"rent nothing / not a service"** posture | **This project realizes
M3/M4.** Carries $06's owner-gated gates: **DPA/consent for non-owner uploads, collector `md5`
keying, WASB-C3** . |
61:| 07-COMPUTE-ABSTRACTION (archived) | `DeviceContext`/`DeviceProfile` (designed) | WASB
hardcodes `device='cuda'` with **no CPU fallback** ? a portability gap M4 closes. |
67:### 4.1 The core contract (must stay byte/behaviour-identical) `[verified]`
74:### 4.2 The deployment profiles
82:### 4.3 Profile selection (the "startup/setup config") `[design]`
90:### 4.4 Compute placement ? and *why stitch runs on Cloud Run* `[design]`
93:### 4.5 Reuse map ? most of the seams already exist `[verified]`
96:### 4.6 The data-flywheel harness (D12 / M11) `[design]`
102:This is the consent?capture?eval?goldenize **flywheel** (the cloud realization of the
PERSONALIZATION contribution model): every consented free reel makes the owned model better.
Reuses `data_pipeline/`, `backend/eval/`, and the M9 consent gate; **adults-only**, raw-
retention gated by the D10 consent.
110:## 7. Deliverables & progress tracker ? **source of truth**
111:Legend: ? Todo . ? In progress . ? Done . ? Gated. **One small PR per row**, off
`origin/master`, no stacking. Default-OFF where it touches core callers.
113:| ID | Deliverable | Depends | Gated? | Status | PR |
115:| M0 | This design doc + project dir + `runlogs/` + ADR-014 | ? | owner sign-off | ? | **?
APPROVED 2026-06-21** |
116:| M1 | `deployment.profile` + `compute_target` config + preset bundles + precedence/auto-
detect; unit tests (no behaviour change, profile='' default) | M0 | safe | ? |
[#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279) |
117:| M2 | `input_backend`/`input_root`; wire main CLI/API input through `StorageRef.parse_uri`
(local=identity); storage-fake tests | M1 | safe | ? |
[#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280) |
118:| M3 | Configurable output/scratch base ? kill hardcoded `output/`
(`trajectory_hybrid:237,313`, `yolo_hybrid:407`, `gemini`); **DEFAULT stays `output/`** ; golden
```

```

+ stub-E2E byte-identical | M1 | ? default-OFF + Launch Report on flip | ? |
[#282](https://github.com/Khelsutra/badminton-highlight-indexer/pull/282) |
119:| M4 | `DeviceContext` + WASB **CPU-fallback**; unified healthcheck/device wiring | M1 |
safe (cuda default unchanged) | ? | [#283](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/283) |
120:| M5 | **truly-local profile** end-to-end (preset + startup checks); first committed
**runlog** (truly-local sample run) | M2,M3,M4 | owner real-GPU | ? |
[#286](https://github.com/Khelsutra/badminton-highlight-indexer/pull/286) (preset+startup-
checks+L4 parity ?; runlog ? owner GPU) |
121:| M6 | Cloud Run GPU **dispatch shim** (control plane ? job via storage ? re-claim) +
**containerized worker image** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests | M5 | ? owner
(GCP spend) | ? | [#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287)
(shim+image+mocked ?; real build/run = M7 ? owner) |
122:| M7 | **cloud-serving profile** e2e on L4: upload`<2GB` + gdrive/bucket ? detect+stitch on
Cloud Run ? reel via signed URL; **DEPLOY_REPORT** (posterity, sample runs + contact sheet) | M6
| ? owner (spend) | ? | ? |
123:| M8 | Per-job **cost ledger** (v6 migration: `owner_id, gpu_active_seconds, wall_seconds,
bytes_out, cost_usd, cost_breakdown_json`) + **pricebook** + aggregation + `/api/.../cost` | M7
| ? owner (billing) | ? | [#288](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/288) (ledger+pricebook+/cost ? default-OFF; real prices + usage-emission ?
owner/follow-up) |
124:| M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user_id`; DPA/consent
gate wiring | M7 | ? owner (privacy/counsel) | ? | ? |
125:| M10 | `byo_worker` / zero-infra-BYO ? **design-only, DEFERRED** | M8,M9 | ? explicit owner
approval | ? | ? |
126:[Omitted long matching line]
127:| **M12** | **gdrive-api` (OAuth) StorageBackend (D14 Northstar): ** cloud reads the source
from + writes the stitched reel back to the user's Google Drive next to the source** (per-user
OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount
backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? |
136:5. ? **DPA/consent ? D10** (free-tier data-for-reels trade; adults-only; derived-data;
counsel-gated to M9).
137:6. ? **Quality floor ? D11** + the **data-flywheel harness D12 / M11** (Gemini-ON until the
owned model clears the floor; capture?store?eval?goldenize meanwhile).
141:7. ? **RESOLVED ? D14:** cloud needs a **`gdrive-api` (OAuth) StorageBackend ** (read the
source from + write the reel back to the user's Drive) ? the Northstar's Drive round-trip ?
distinct from the local **mount** backend (which stays for truly-local). New milestone **M12**.
147:- [ ] M1?M4 land (config + input + output-base + device layer), each green on golden cross-
OS + experiment no-regression.
149:- [ ] **truly-local** (M5) and **cloud-serving** (M7) each have a committed **runlog /
DEPLOY_REPORT** with a real sample run.
150:- [ ] Cost ledger (M8) makes GPU + **stitch** + egress cost visible per job; edge auth (M9)
gates multi-tenant.
151:- [ ] **Data-flywheel harness (M11)** captures?stores?shadow-evals?goldenizes consented
adult jobs; the owned model is on a measured path to the D11 floor.
158:### Changelog
159:- 2026-06-21 ? created from the design workflow output; M0 in progress.
160:- 2026-06-21 ? **M0 APPROVED**; **M1 landed**
([#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279)):
`deployment.profile` + `compute_target` + preset bundles + precedence/auto-detect-as-suggestion
(D15) on `AppConfig`. Pure/additive, `profile=''` default == today's behaviour (zero core
change); presets touch only existing knobs (no M2 `input_backend` / M3 output-base over-reach).
Suite 1068 green, cov 84.11%.
161:- 2026-06-21 ? **M2 landed** ([#280](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/280)): `input_backend`/`input_root` + URI-aware main API/CLI input via
`resolve_input_uri`/`acquire_local_input` (local = identity passthrough, byte-for-byte
unchanged; bucket/Drive fetched to scratch). Unsupported scheme ? clean 400; `local://` accepted
on the resolved key; hosted-mode rejects non-local URIs; source URI kept for DB/report
provenance. Suite 1105 green, cov 84.84%.
162:- 2026-06-21 ? **M3 landed** ([#282](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/282)): configurable output/scratch base via
`backend/utils/paths.output_base(config)` ? the 3 segmenters' chunk/handoff/temp dirs root under
`output.output_dir` (default `"output"`, byte-identical; default-OFF ? no Launch Report). Fixes
the latent bug where scratch ignored `--output-dir`. Suite 1114 green, cov ~84.9%.
163:[Omitted long matching line]

```

164:[Omitted long matching line]
165:[Omitted long matching line]
166:[Omitted long matching line]
167:[Omitted long matching line]

14. assistant (2026-06-21T13:57:06.264Z)

The \$7 tracker on disk fully confirms the dossier's milestone statuses. I have verified the decisive claims directly: the wiring gap (API path bypasses the compute seam), the default-OFF factory, the tracker status, and the actual git state (I am on `feat/gcp-ops-agent-and-dryrun-guide`, local master == origin/master at #289, README ahead reflecting M5-M8). Let me do one final check on the active-session evidence: the open M9 PR #290 and the timestamps.

15. user (2026-06-21T13:57:08.973Z)

```
[{"headRefName":"feat/gcp-ops-agent-and-dryrun-guide","isDraft":false,"number":291,"title":"feat(gcp): always-install Ops Agent on GPU VMs + a cloud-GPU dry-run guide","updatedAt":"2026-06-21T13:56:19Z"}, {"headRefName":"feat/serving-m9-auth","isDraft":false,"number":290,"title":"feat(serving): M9-auth ? Cloudflare Access edge-auth + per-tenant owner_id (default-OFF)","updatedAt":"2026-06-21T13:49:56Z"}]
```

16. assistant (2026-06-21T13:57:12.974Z)

Confirmed live: PR #290 (M9-auth) is open and non-draft, updated 13:49Z today, on branch `feat/serving-m9-auth`. PR #291 (the branch I'm sitting on) is the GCP ops-agent/dry-run companion, also open. The serving track is actively being pushed right now. I have everything verified directly against code, the tracker, and live PR state. Submitting the synthesis.

17. user (2026-06-21T13:58:11.589Z)

Output does not match required schema: root: must NOT have additional properties

18. assistant (2026-06-21T13:58:15.383Z)

The error indicates an additional property. The schema requires exactly the six named properties. Let me resubmit with only those properties (I had no extra keys, but the tool may have appended `title`/`loading\_messages` style fields ? re-submitting the exact six required fields).

19. user (2026-06-21T13:58:58.201Z)

Structured output provided successfully

20. assistant (2026-06-21T13:59:12.160Z)

Structured output submitted successfully. Summary of what I verified directly (not just from the dossier):

- ****Git state****: I am on branch `feat/gcp-ops-agent-and-dryrun-guide` (HEAD `c3dbd00`); local `master` == `origin/master` == `e68b583` (#289). The checkout is NOT stale. The on-disk `docs/COMPUTE\_DECOUPLED\_SERVING/README.md` \$7 tracker reflects M5?M8 landed and is authoritative.
- ****Decisive wiring gap confirmed****: `get\_compute\_backend` is referenced only in `backend/worker/\_\_main\_\_.py` (line 166); `backend/main.py::execute\_processing` and `backend/api/job\_worker.py` have zero references ? so every `/api/process` job runs in-process and the Cloud Run path is reachable only via the `backend.worker` CLI.
- ****Default-OFF factory confirmed**** in `backend/compute/\_\_init\_\_.py` (returns `None` for all targets except `cloud\_run`).
- ****Active work confirmed live****: `gh pr list` shows PR #290 (M9-auth) open, non-draft, branch `feat/serving-m9-auth`, updated 2026-06-21T13:49Z.

Verdict delivered: the Cloud-Run-friendly work lives in the `docs/COMPUTE\_DECOUPLED\_SERVING/` subproject + `backend/compute/` + `deploy/cloudrun/`, and it is decisively being actively worked by the concurrent serving session right now.